

TREES

Data Structures
(FAANG Cheat Sheet)

1. TREE FOUNDATIONS

TREE vs BINARY TREE vs BST

Tree

Binary Tree

Binary Search Tree

A hierarchical data structure. Each node has at most 2 children. Left < Root < Right

TYPES OF BINARY TREES

Full Binary Tree, Complete Binary Tree, Perfect Binary Tree, Balanced Binary Tree.

KEY TERMINOLOGY

- Node: Basic unit
- Root: Topmost node
- Parent, Child, Leaf, Internal Node, Edge, Subtree
- Height, Depth, Level

2. TRAVERSALS

DEPTH FIRST SEARCH (DFS)

Preorder (Root → Left → Right)

Inorder (Left → Root → Right)

Postorder (Left → Right → Root)

BREADTH FIRST SEARCH (BFS)

Level Order Traversal

6. BST BIBLE

BST RULE

Left Subtree < Root < Right Subtree

OPERATIONS IN BST

- Search
- Insert
- Delete

3. TREE RECURSION TEMPLATE

THE RECURSION FLOW

- Base Case
- Faith
- Recur Left
- Recur Right
- Combine
- Return

RETURN TYPE DECIDES RECURSION

Return Type	What it Means
int	Return some value
boolean	Return true/false

4. DFS PATTERN PROBLEMS

- MAX DEPTH
- SAME TREE
- INVERT TREE
- DIAMETER OF BINARY TREE
- BALANCED BINARY TREE

5. BFS PATTERN PROBLEMS

BFS TEMPLATE (LEVEL ORDER TRAVERSAL)

```
Queue<TreeNode> q = new LinkedList<>();
q.offer(root);
while (!q.isEmpty()) {
    int size = q.size();
    for(int i=0; i<size; i++) {
        TreeNode node = q.poll();
        if(node.left != null)
            q.offer(node.left);
        if(node.right != null)
            q.offer(node.right);
    }
}
```

7. ADVANCED TREE PROBLEMS

- LCA in Binary Tree
- Build Tree from Traversals
- Path Sum
- Serialize/Deserialize

8. TREE - ONE PAGE REVISION

TRAVERSAL REMINDERS

Preorder, Inorder, Postorder, BFS

KEY AHA MOMENTS

- Max Depth → 1 + max(left, right)
- Same Tree → left && right
- Invert Tree → Swap then recurse

TREE FOUNDATIONS

The base you must be 100% clear on.
(FAANG Cheat Sheet)

1

1.1 TREE vs BINARY TREE vs BINARY SEARCH TREE (BST)

TREE

- A hierarchical structure.
- Each node can have 0 or more children.
- No ordering between child nodes.

```
graph TD
  style A stroke:#16a34a,fill:white
  style B stroke:#16a34a,fill:white
  style C stroke:#16a34a,fill:white
  style D stroke:#16a34a,fill:white
  A --> B
  A --> C
  A --> D
```

BINARY TREE

- Each node has at most 2 children: Left and Right.
- No ordering between left and right subtree.

```
graph TD
  style A stroke:#7e22ce,fill:white
  style B stroke:#7e22ce,fill:white
  style C stroke:#7e22ce,fill:white
  style D stroke:#7e22ce,fill:white
  style E stroke:#7e22ce,fill:white
  style F stroke:#7e22ce,fill:white
  A --> B
  A --> C
  B --> D
  C --> E
  D --> F
```

BINARY SEARCH TREE (BST)

- Binary Tree + Left < Root < Right for every node.
- Inorder traversal gives sorted order.

```
graph TD
  style A stroke:#ea580c,fill:white
  style B stroke:#ea580c,fill:white
  style C stroke:#ea580c,fill:white
  style D stroke:#ea580c,fill:white
  style E stroke:#ea580c,fill:white
  style F stroke:#ea580c,fill:white
  A --> B
  A --> C
  B --> D
  C --> E
  D --> F
```

1.2 TYPES OF BINARY TREES

FULL BINARY TREE

Every node has 0 or 2 children.

```
graph TD
  style A stroke:#16a34a,fill:white
  style B stroke:#16a34a,fill:white
  style C stroke:#16a34a,fill:white
  style D stroke:#16a34a,fill:white
  style E stroke:#16a34a,fill:white
  style F stroke:#16a34a,fill:white
  style G stroke:#16a34a,fill:white
  A --> B
  A --> C
  B --> D
  B --> E
  C --> F
  C --> G
```

COMPLETE BINARY TREE

All levels are completely filled except possibly the last level, and all nodes are as far left as possible.

```
graph TD
  style A stroke:#7e22ce,fill:white
  style B stroke:#7e22ce,fill:white
  style C stroke:#7e22ce,fill:white
  style D stroke:#7e22ce,fill:white
  style E stroke:#7e22ce,fill:white
  style F stroke:#7e22ce,fill:white
  A --> B
  A --> C
  B --> D
  C --> E
  D --> F
```

PERFECT BINARY TREE

All internal nodes have 2 children and all leaves are at the same level.

```
graph TD
  style A stroke:#ea580c,fill:white
  style B stroke:#ea580c,fill:white
  style C stroke:#ea580c,fill:white
  style D stroke:#ea580c,fill:white
  style E stroke:#ea580c,fill:white
  style F stroke:#ea580c,fill:white
  style G stroke:#ea580c,fill:white
  A --> B
  A --> C
  B --> D
  B --> E
  C --> F
  C --> G
```

BALANCED BINARY TREE

For every node, $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$

```
graph TD
  style A stroke:#0284c7,fill:white
  style B stroke:#0284c7,fill:white
  style C stroke:#0284c7,fill:white
  style D stroke:#0284c7,fill:white
  style E stroke:#0284c7,fill:white
  style F stroke:#0284c7,fill:white
  style G stroke:#0284c7,fill:white
  A --> B
  A --> C
  B --> D
  C --> E
  D --> F
  E --> G
```

SKewed BINARY TREE

Every node has only one child. (Left Skewed or Right Skewed)

```
graph TD
  style A stroke:#dc2626,fill:white
  style B stroke:#dc2626,fill:white
  style C stroke:#dc2626,fill:white
  style D stroke:#dc2626,fill:white
  style E stroke:#dc2626,fill:white
  style F stroke:#dc2626,fill:white
  style G stroke:#dc2626,fill:white
  A --> B
  B --> C
  C --> D
  D --> E
  E --> F
  F --> G
```

1.3 KEY TERMINOLOGY

	Node	: Basic unit of a tree.			
	Root	: Topmost node of the tree.			
	Edge	: Connection between two nodes.			
graph TD A((A)):::highlight → B((B)) A → C((C)) classDef highlight stroke:#7e22ce,stroke-width:2px;	Parent	: Immediate ancestor of a node.			
graph TD A((A)) → B((B)):::highlight classDef highlight stroke:#7e22ce,stroke-width:2px;	Child	: Immediate descendant of a node.			
	Leaf Node	: Node with no children.			
graph TD A(()) → B(()):::highlight B → C(()) classDef highlight stroke:#ea580c,stroke-width:2px;	Internal Node	: Node with at least one child.			
graph TD A((A)) → B((B)):::highlight A → C((C)):::highlight classDef highlight stroke:#0284c7,stroke-width:2px;	Siblings	: Children of the same parent.			
graph TD A((A)) -.→ B((B))	Depth of Node	: Number of edges from root to the node. (Root depth = 0)			
graph TD A((A)) → B((B)) B --> D((D)) A → C((C)) B → E((E))	Level of Node	: Depth + 1 (Root level = 1)			
graph TD A((A)):::highlight → B((B)):::highlight classDef highlight stroke:#16a34a,stroke-width:2px;	Height of Node	: Number of edges on the longest path from the node to a leaf.			
graph TD A((A)):::highlight → B((B)):::highlight classDef highlight stroke:#16a34a,stroke-width:2px;	Height of Tree	: Height of the root node. (Longest path from root to a leaf)			
graph LR A((A)):::highlight → B((B)):::highlight B → C((C)):::highlight classDef highlight stroke:#7e22ce,stroke-width:2px;	Path	: Sequence of nodes connected by edges.			
graph TD A(()) → B(()) A → C(()) classDef default stroke:#ea580c;	Subtree	: A tree formed by a node and its descendants.			



Remember: Tree is a hierarchical structure. Understanding these basics makes every tree problem easier!



2 TRAVERSALS

Ways to visit every node of a tree

QUICK VIEW

graph TD A((1)) --> B((2)) A --> C((3)) B --> D((4)) B --> E((5)) C --> F((6)) C --> G((7)) classDef root fill:#f0fdf4,stroke:#22c55e,stroke-width:2px class 1 root classDef level1 fill:#fff2cc,stroke:#ffc107,stroke-width:2px class 2,3 level1 classDef level2 fill:#fff2cc,stroke:#ffc107,stroke-width:2px class 4,5,6,7 level2

① = Root
2, 3 = Level 1
4, 5, 6, 7 = Level 2

2.1 DEPTH FIRST SEARCH (DFS) TRAVERSALS

1 PREORDER (Root → Left → Right)

graph TD A((1)):::root --> B((2)) A --> C((3)) B --> D((4)) B --> E((5)) C --> F((6)) C --> G((7)) classDef root fill:#f0fdf4,stroke:#22c55e,stroke-width:2px

ORDER OF VISIT

1 - 2 - 4 - 5 - 3 - 6 - 7

AHA MOMENT

Need parent first.
Visit root before its children.

WHEN TO USE?

- Create / Copy / Clone Tree
- Serialize Tree
- Prefix expressions
- Build Tree (with Inorder)

2 INORDER (Left → Root → Right)

graph TD A((1)):::root --> B((2)) A --> C((3)) B --> D((4)) B --> E((5)) C --> F((6)) C --> G((7)) classDef root fill:#fff2cc,stroke:#ffc107,stroke-width:2px

ORDER OF VISIT

4 - 2 - 5 - 1 - 6 - 3 - 7

AHA MOMENT

In BST, Inorder gives
sorted order.

WHEN TO USE?

- BST Problems
- Sorted order
- Kth Smallest / Largest in BST
- Validate BST (Iterative)

3 POSTORDER (Left → Right → Root)

graph TD A((1)):::root --> B((2)) A --> C((3)) B --> D((4)) B --> E((5)) C --> F((6)) C --> G((7)) classDef root fill:#fff2cc,stroke:#ffc107,stroke-width:2px

ORDER OF VISIT

4 - 5 - 2 - 6 - 7 - 3 - 1

AHA MOMENT

Need children's answers first.
Process children before parent.

WHEN TO USE?

- Delete / Free Tree
- Postfix expressions
- Evaluate expressions
- Height, Diameter, Balanced, LCA, Path Sum (etc.)

2.2 BREADTH FIRST SEARCH (BFS) TRAVERSAL (LEVEL ORDER)

```
graph TD
    A((1)) --- root
    root --> B((2))
    root --> C((3))
    root --> D((4))
    root --> E((5))
    root --> F((6))
    root --> G((7))
    classDef root fill:#ffffaf,stroke:#ea580c,stroke-width:2px
    class root root
```

Level 1

Level 2

ORDER OF VISIT (LEVEL ORDER)

1 - 2 - 3 - 4 - 5 - 6 - 7

AHA MOMENT

Need level by level.
Use Queue.
Queue size tells current level.

TEMPLATE (BFS LEVEL ORDER)

```
Queue<Node> q = new LinkedList<>();
q.offer(root);

while (!q.isEmpty()) {
    int size = q.size(); // nodes in current level
    for (int i = 0; i < size; i++) {
        Node node = q.poll();
        // process node
        if (node.left != null)
            q.offer(node.left);
        if (node.right != null)
            q.offer(node.right);
    }
}
```

2.3 DFS vs BFS

	DFS (Depth First Search)	BFS (Breadth First Search)
 Strategy	Go deep to a leaf, then backtrack	Visit level by level from left to right
 Data Structure	Stack (or Recursion)	Queue
 Time Complexity	$O(n)$	$O(n)$
 Space Complexity	$O(h)$ (h = height of tree)	$O(w)$ (w = max width of tree)
 Best For	Path, Height, DP on trees, Backtracking	Shortest path in unweighted tree, Level wise problems

2.4 WHEN TO USE WHICH TRAVERSAL?

	Need parent before children?	→	Use Preorder
	Need sorted order (BST)?	→	Use Inorder
	Need children's results first?	→	Use Postorder
	Need level by level?	→	Use BFS
	Modify / Build / Serialize Tree?	→	Mostly Preorder
	Delete / Free / Evaluate?	→	Mostly Postorder

**FINAL MANTRA**

- Tree problems → **Think DFS first.**
- Level problems → **Think BFS first.**
- Understand the need, then choose the traversal.

n = number of nodes
 h = height of tree
 w = maximum width of tree

3 TREE RECURSION TEMPLATE

Master this template, solve any tree problem!

3.1 THE RECURSION FLOW (FAITH → RECUR → COMBINE → RETURN)

1	BASE CASE	Check and handle the base condition.	<pre>if (node == null) return ...;</pre>
2	FAITH	Assume left and right subtrees will give correct answers.	
3	RECUR LEFT	Recur for the left subtree.	<pre>left = solve(node.left);</pre>
4	RECUR RIGHT	Recur for the right subtree.	<pre>right = solve(node.right);</pre>
5	COMBINE	Use left and right results. Do the required work.	<pre>ans = ...;</pre>
6	RETURN	Return the final answer.	<pre>return ans;</pre>

UNIVERSAL TEMPLATE

```
Type solve(Node* node) {
    if (node == null) {
        return ...; // Base Case
    }

    Type left = solve(node.left); // Recur Left
    Type right = solve(node.right); // Recur Right

    Type ans = ...; // Combine
    return ans; // Return
}
```

KEY IDEA

- Break the problem into left and right subproblems.
- Solve both (faith).
- Combine their results.
- Return the final answer up the recursion stack.

★ **FAITH** → You assume your recursive calls (left & right) are correct and return the right answer.

3.2 RETURN TYPE DECIDES THE RECURSION

Return Type	What it Means	Examples
int	Return some value (height, count, sum, etc.)	Max Depth, Diameter, Sum of Nodes
boolean	Return true / false	Same Tree, Balanced Tree, Valid BST
TreeNode*	Return a node	LCA, Invert Tree, Search in BST
List / Vector	Return a list of values / nodes	All Paths, Level Order, Right Side View
void	Do something (print / modify)	Print Traversals, Populate Result

3.3 WHICH TRAVERSAL TO USE?

Goal	Need	Traversal	Why?
Need parent before children?	Parent info first	Preorder (Root → Left → Right)	Visit root before going to children.
Need sorted order (in BST)?	Left first, then root, then right	Inorder (Left → Root → Right)	In BST, Inorder gives sorted order.
Need children's answers first?	Children results before parent	Postorder (Left → Right → Root)	Process children before the root.
Need level by level?	Level wise processing	BFS (Level Order)	Visit nodes level by level using Queue.

3.5 COMMON BASE CASES

<pre>if (node == null) return ... ;</pre>	Used in almost all problems. Empty subtree.
<pre>if (node.left == null && node.right == null) return ... ;</pre>	Leaf node condition.
<pre>if (node == target) return ... ;</pre>	Found the target node.
<pre>if (k == 0) return ... ;</pre>	Base case for depth / distance based problems.
<pre>if (low > high) return ... ;</pre>	Used in tree construction from array / traversal.

- ✓ 1. Draw the tree and try dry run.
- ✓ 2. Identify what you need at each node.
- ✓ 3. Decide return type.
- ✓ 4. Write the template first.
- ✓ 5. Fill the combine logic.
- ✓ 6. Dry run again and verify.

Think small,
Recur small,
Combine smartly,
Return clearly.



- 
- FINAL MANTRA**

7/19

4 DFS PATTERN PROBLEMS (POSTORDER FOCUSED)

Most important tree problems follow DFS (Postorder). Master these patterns!

💡 AHA = Key Insight

⚙️ Pattern = How to think

Σ Formula = How to compute

🕒 Complexity = Time & Space

1. MAX DEPTH OF BINARY TREE

Example

graph TD 1((1)) → 2((2)) 1 → 3((3)) 2 → 4((4)) 2 → 5((5)) 3 → 6((6))

Output: 3

💡 AHA

Need the height of left and right subtrees first, then take max. That's Postorder.

⚙️ Pattern

Postorder
(Left → Right → Root)

Σ Formula

$1 + \max(\text{left}, \text{right})$

Base Case:
if (node == null)
return 0;

Code (Java)

```
int maxDepth(TreeNode root) {
    if (root == null) return 0;
    // Base Case
    int left = maxDepth(root.left); // Left
    int right = maxDepth(root.right); // Right
    return 1 + Math.max(left, right); // Combine
}
```

Complexity

Time: **O(n)**
Space: **O(h)**
(h = height of tree)

2. SAME TREE

Example

p q
graph TD p1((1)):::pNode - graph TD q1((1)):::qNode -
→ p2((2)):::pNode p1 → → q2((2)):::qNode q1 →
p3((3)):::pNode p2 → q3((3)):::qNode q2 →
p4((4)):::pNode classDef q4((4)):::qNode classDef
pNode qNode
fill:#fff,stroke:#333,stroke-fill:#fff,stroke:#333,stroke-
width:1px; width:1px;

Output: true

💡 AHA

Both trees must match at current node, left subtree and right subtree. Return boolean.

⚙️ Pattern

Postorder
(Left → Right → Root)

Σ Formula

Left && Right

Base Case:
Both null → true
One null → false
Values not equal → false

Code (Java)

```
boolean isSameTree(TreeNode p, TreeNode q) {
    if (p == null && q == null) return true;
    if (p == null || q == null) return false;
    if (p.val != q.val) return false;
    boolean left = isSameTree(p.left, q.left);
    boolean right = isSameTree(p.right, q.right);
    return left && right;
    // Combine
}
```

Complexity

Time: **O(n)**
Space: **O(h)**

3. INVERT BINARY TREE

Example

Input Output
graph TD graph TD
I1((1)) → O1((1)) →
I2((2)) I1 → → O3((3)) O1 →
I3((3)) I2 → O2((2)) O2 --
I4((4)) I2 -- > O5((5)) O2
> I5((5)) → O4((4))

💡 AHA

Need to process parent first, then swap children and recurse. That's Preorder.

⚙️ Pattern

Preorder
(Root → Left → Right)

Σ Formula

Swap Left and Right

Base Case:
if (node == null)
return null;

Code (Java)

```
TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    // Base Case
    TreeNode temp = root.left;
    // Swap
    root.left = root.right;
    root.right = temp;
    invertTree(root.left);
    // Left
    invertTree(root.right);
    // Right
    return root;
}
```

Complexity

Time: **O(n)**
Space: **O(h)**

4. DIAMETER OF BINARY TREE

Example

graph TD 1((1)) → 2((2)) 1 → 3((3)) 2 → 4((4)) 2 → 5((5))

Output: 3
(Path: 4 → 2 → 1 → 3)

💡 AHA

Diameter can pass through any node. Need left height and right height at every node. Update global answer.

⚙️ Pattern

Postorder
(Left → Right → Root)

Σ Formula

diameter =
max(diameter,
left + right)

return height =
1 + max(left,
right)

```
int diameter = 0; // global
int diameterOfBinaryTree(TreeNode root) {
    height(root);
    return diameter;
}
int height(TreeNode node) {
    if (node == null) return 0;
    int left = height(node.left);
    int right = height(node.right);
    diameter = Math.max(diameter, left + right); // update
    return 1 + Math.max(left, right); // height
}
```

Complexity

Time: **O(n)**
Space: **O(h)**

5. BALANCED BINARY TREE

Example

graph TD 1((1)) → 2((2)) 1 → 3((3)) 2 → 4((4)) 2 → 5((5))

Output: true

💡 AHA

Every node must have left and right subtree heights difference ≤ 1. Need height of subtrees.

⚙️ Pattern

Postorder
(Left → Right → Root)

Σ Formula

If |left - right| > 1
return -1
(imbalance found)

Else return
1 + max(left,
right)

```
int isBalanced(TreeNode root) {
    return height(root) != -1;
}
int height(TreeNode node) {
    if (node == null) return 0;
    int left = height(node.left);
    if (left == -1) return -1;
    int right = height(node.right);
    if (right == -1) return -1;
    if (Math.abs(left - right) > 1)
        return -1;
    return 1 + Math.max(left, right);
}
```

Complexity

Time: **O(n)**
Space: **O(h)**



POSTORDER - WHEN TO USE?

- Need children's answers first.
- Need height / depth / size.
- Need info from left & right to compute current node.
- Delete / Free / Evaluate expressions.
- Diameter, Balanced, Max Path Sum, LCA (Binary Tree) etc.

QUICK RECAP

Height / Depth	1 + max(left, right)
Diameter	left + right (update ans)
Balanced	abs(left - right) ≤ 1
Same Tree	left == right
Invert Tree	Swap + Recur



COMPLEXITY LEGEND

n = number of nodes
h = height of tree
(For balanced tree: h = log n)
(For skewed tree: h = n)

★ **FINAL MANTRA:** Identify pattern → Write template → Apply formula → Handle base cases → Return correct type

5 BFS PATTERN PROBLEMS (LEVEL ORDER FOCUSED)

(FAANG Cheat Sheet)

PAGE 5

BFS = Level by Level using Queue

💡 AHA = Key Insight

⚙️ Pattern = How to think

Σ Formula / Idea

🕒 Complexity = Time & Space

1. LEVEL ORDER TRAVERSAL (LC 102)

Example

graph TD 1((1)) → 2((2)) 1 --> 3((3)) 2 → 4((4)) 2 → 5((5)) 3 → 6((6)) 3 → 7((7))

Output:

[[1], [2,3], [4,5,6,7]]

💡 AHA

Need nodes level by level.
Use Queue.
Queue size tells current level.

⚙️ Pattern

BFS (Level Order) using Queue

[Σ Idea / Formula](#)

Process all nodes in queue
(size = current level). Add
children to queue.

Code (Java)

```
List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> res = new ArrayList<>();
    if (root == null) return res;

    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);

    while (!q.isEmpty()) {
        int size = q.size();
        List<Integer> level = new ArrayList<>();

        // process current level
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            level.add(node.val);

            if (node.left != null)
                q.offer(node.left);
            if (node.right != null)
                q.offer(node.right);
        }
        res.add(level);
    }
    return res;
}
```

🕒 Complexity

Time: O(n)

Every node visited once.

Space: O(n)

Queue can hold max nodes of
last level.

2. RIGHT SIDE VIEW (LC 199)

Example

graph TD 1((1)) → 2((2)) 1 → 3((3)) 2 → 4((4)) 2 → 5((5)) 3 → 6((6)) 3 → 7((7))

Output: [1, 3, 7]
(Rightmost of each level)

⚡ AHA

We need the last node of each level. In each level, the last node we pop is the rightmost.

⚙️ Pattern

BFS (Level Order)

Σ Idea

In each level, pick the last node's value.

Code (Java)

```
List<Integer>
rightSideView(TreeNode root)
{
    List<Integer> res = new
    ArrayList<>();
    if (root == null) return
    res;
    Queue<TreeNode> q = new
    LinkedList<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i <
        size; i++) {
            TreeNode node =
            q.poll();
            if (i == size -
            1) res.add(node.val); //
            last node
            if (node.left !=
            null) q.offer(node.left);
            if (node.right
            != null)
            q.offer(node.right);
        }
        return res;
    }
}
```

3. ZIGZAG LEVEL ORDER (LC 103)

Example

graph TD 1((1)) → 2((2)) 1 → 3((3)) 2 → 4((4)) 2 → 5((5)) 3 → 6((6)) 3 → 7((7))

Output:
[[1], [3,2], [4,5,6,7]]

⚡ AHA

Direction changes at every level. Use a flag.

⚙️ Pattern

BFS + Direction Toggle

Σ Idea

If leftToRight →
left to right
Else →
right to left

Code (Java)

```
List<List<Integer>>
zigzagLevelOrder(TreeNode root)
{
    List<List<Integer>> res =
    new ArrayList<>();
    if (root == null) return
    res;
    Queue<TreeNode> q = new
    LinkedList<>();
    q.offer(root);
    boolean leftToRight = true;
    while (!q.isEmpty()) {
        int size = q.size();
        LinkedList<Integer>
        level = new LinkedList<>();
        for (int i = 0; i <
        size; i++) {
            TreeNode node =
            q.poll();
            if (leftToRight)
            level.addLast(node.val);
            else
            level.addFirst(node.val);

            if (node.left !=
            null) q.offer(node.left);
            if (node.right !=
            null) q.offer(node.right);
        }
        res.add(level);
        leftToRight =
        !leftToRight; // toggle
    }
    return res;
}
```

4. BFS TEMPLATE (REMEMBER THIS!)

```
// BFS Template
if (root == null) return;

Queue<TreeNode> q = new LinkedList<>();
q.offer(root);

while (!q.isEmpty()) {
    int size = q.size(); // nodes in
    current level

    for (int i = 0; i < size; i++) {
        TreeNode node = q.poll();
        // — Do something with node
        here —

        if (node.left != null)
        q.offer(node.left);
        if (node.right != null)
        q.offer(node.right);
    } // — Level completed —
}
```

Key Points

✓ Use Queue

✓ **queue.size() = current level size**

✓ Process level by level

✓ Add children for next level

5. HOW QUEUE WORKS IN BFS

Step	Queue (Front → Back)	Level	Action	Visual Flow
1	[1]	0	Start: push root	graph TD subgraph Level 0 1((1)) end subgraph Level 1 2((2)) 3((3)) end subgraph Level 2 4((4)) 5((5)) 6((6)) 7((7)) end
2	[] → [2, 3]	1	Pop 1, push 2, 3	
3	[2, 3]	1	Process level 1	
4	[] → [4, 5, 6, 7]	2	Pop 2, push 4, 5 Pop 3, push 6, 7	
5	[4, 5, 6, 7]	2	Process level 2	
6	[]	3	All nodes processed	
7	[]	-	Queue empty, done!	

WHEN TO USE BFS?

- ✓ Need level by level processing
- ✓ Shortest path in unweighted graphs
- ✓ Minimum steps / transformations
- ✓ Problems involving distance / levels
- ✓ Tree view from top / side / bottom

BFS vs DFS (Quick Comparison)

Feature	BFS	DFS
Strategy	Level by Level	Go Deep, then Backtrack
Data Structure	Queue	Stack / Recursion
Best For	Level problems	Path / Depth problems

QUICK RECAP

- ★ Use Queue for BFS.
- ★ queue.size() gives current level.
- ★ Process all nodes of a level together.
- ★ Add children for next level.
- ★ BFS = Clean + Level Order + Optimal (for shortest path problems).

Space	O(max width)	O(max depth)
-------	--------------	--------------

🏆 FINAL MANTRA: Think Level by Level → Use Queue → Process Level → Add Children → Repeat → Get Answer

BST BIBLE (BINARY SEARCH TREE)

The most important BST rules, operations and patterns

(FAANG Cheat Sheet)

PAGE
6

1 BST RULE

For every node N in BST

All values in Left Subtree < N < All values in Right Subtree

Example

All graph TD 8((8)) → 3((3)) 8 → 10((10)) 3 → 1((1)) 3 → 6((6)) 10 → 14((14))

- ✓ Left subtree → smaller values
- ✓ Right subtree → larger values
- ✓ Inorder traversal gives sorted order

2 SEARCH IN BST

Idea: Use BST property to decide the direction.

```
if key = root.val → found
else if key < root.val → go left
else → go right
```

Example

graph TD 8((8)) → 3((3)) 8 → 10((10)) 3 → 1((1)) 3 → 6((6)):::highlight 10 → 14((14))

classDef highlight stroke:#ef4444,stroke-width:2px,stroke-dasharray: 5 5

Search:
6

8 > 6 → go left
3 < 6 → go right
6 == 6 → found

Complexity

Time: **O(h)** Space: **O(1)** iterative / **O(h)** recursive
(h = height of tree)

3 INSERT INTO BST

Idea: Find correct position and insert.

Steps:

1. If tree is empty → new node becomes root
2. Else compare and go left or right
3. Insert when NULL is found

Example: Insert 7

graph TD 8((8)) → 3((3)) 8 → 10((10)) 3 → 1((1)) 3 → 6((6)) 10 → 14((14)) 6 → empty1[] 6 → 7((7)):::newnode style empty1 fill:none,stroke:none classDef newnode stroke:#3b82f6,stroke-width:2px,stroke-dasharray: 5 5

Complexity

Time: **O(h)** Space: **O(1)** iterative / **O(h)** recursive

4 DELETE FROM BST

Cases to handle when deleting node X:

Case 1: X is a leaf node

→ Just delete it.

```
graph TD
  6((6)) --> 5((5))
  6 --> 8((8))
  8 --> 10((10))
  10 --> 3((3))
  3 --> 7((7))
  7 --> 9((9))
  9 --> 14((14))
  style 6 stroke:#ea580c,stroke-width:2px,stroke-dasharray: 5 5
  style 8 stroke:#ea580c,stroke-width:2px,stroke-dasharray: 5 5
```

Case 2: X has one child

→ Replace X with its child.

```
graph TD
  6((6)) --> 5((5))
  6 --> 8((8))
  8 --> 10((10))
  10 --> 3((3))
  3 --> 7((7))
  7 --> 9((9))
  9 --> 14((14))
  style 6 fill:none,stroke:none
  style 8 fill:none,stroke:none
  classDef delete stroke:#ea580c,stroke-width:2px,stroke-dasharray: 5 5
  class 6,8 delete
```

Case 3: X has two children

→ Replace X with Inorder Successor (smallest in right subtree) or Inorder Predecessor (largest in left subtree).

```
graph TD
  6((6)) --> 5((5))
  6 --> 8((8))
  8 --> 10((10))
  10 --> 3((3))
  3 --> 7((7))
  7 --> 9((9))
  9 --> 14((14))
  style 6 fill:none,stroke:none
  style 8 fill:none,stroke:none
  classDef delete stroke:#ea580c,stroke-width:2px,stroke-dasharray: 5 5
  class 6,8 delete
  classDef succ stroke:#3b82f6,stroke-width:2px,stroke-dasharray: 5 5
  class 10 succ
```

Complexity

Time: **O(h)** Space: **O(1)** iterative / **O(h)** recursive

5 Kth SMALLEST ELEMENT (LC 230)

Idea: Inorder traversal of BST gives sorted order.

Steps:

1. Do Inorder (Left → Root → Right)
2. Count nodes
3. When count == k → return node

Example

```
graph TD
  8((8)) --> 3((3))
  8 --> 10((10))
  10 --> 3((3))
  3 --> 7((7))
  7 --> 9((9))
  9 --> 14((14))
  style 8 stroke:#2563eb,stroke-width:2px
```

Inorder:
1, 3, 6, 8, 10, 14

k = 3
→
Answer = 6

Complexity

Time: **O(h + k)** Space: **O(h)**
(h = height, k = position of answer)

6 VALIDATE BST (LC 98)

Idea: Each node must lie in a valid range.

For every node:
 $\text{min} < \text{node.val} < \text{max}$

Steps:

1. Use two bounds: min and max
2. Check: $\text{min} < \text{node.val} < \text{max}$
3. Recur Left → (min, node.val)
4. Recur Right → (node.val, max)

Example

```
graph TD
  8((8)) --> 3((3))
  8 --> 10((10))
  10 --> 3((3))
  3 --> 7((7))
  7 --> 9((9))
  9 --> 14((14))
  style 8 stroke:#2563eb,stroke-width:2px
```

For node 6:
Range (3, 8)
✓

For node 14:
Range (10, +∞)
✓

All nodes satisfy → Valid BST

Complexity

Time: **O(n)** Space: **O(h)**

7 LCA IN BST (LC 235)

Idea: Use BST property to find LCA.

If both p and q < root → go left
If both p and q > root → go right
Else → root is LCA

Example: p = 1, q = 14

graph TD 8((8)):::lca → 3((3)) 8 → 10((10)) 3 → 1((1)):::pq 3 → 6((6)) 10 → 14((14)):::pq classDef lca fill:#fca5a5,stroke:#b91c1c,stroke-width:2px classDef pq fill:#fff,stroke:#000,stroke-width:2px,stroke-dasharray: 5 5 8 is LCA

Complexity

Time: O(h) Space: O(1) iterative / O(h) recursive

BST CHEAT SHEET

Operation	Idea	Time	Space
Search	Use BST property	O(h)	O(1) / O(h)
Insert	Find position & insert	O(h)	O(1) / O(h)
Delete	Handle 3 cases	O(h)	O(1) / O(h)
Kth Smallest	Inorder + count	O(h + k)	O(h)
Validate BST	Use range (min, max)	O(n)	O(h)
LCA (BST)	Use BST property	O(h)	O(1) / O(h)

BST PROPERTIES

- ✓ Left < Root < Right
- ✓ Inorder = Sorted
- ✓ No duplicates (by default)
- ✓ Efficient operations: O(h)
- ✓ Balanced BST → O(log n)

★ REMEMBER

- Think in terms of range.
- Inorder is your best friend.
- BST gives you direction.

8 INORDER = SORTED ORDER

Proof by Example

graph TD 8((8)) → 3((3)) 8 → 10((10)) 3 → 1((1)) 3 → 6((6)) 10 → 14((14))

Inorder Traversal
(Left → Root → Right)

= 1, 3, 6, 8, 10, 14
(sorted order)



Why?

All left < root < all right
So visiting Left → Root → Right
gives increasing order.



FINAL MANTRA

Understand Rule
(Left < Root < Right)



Use Property
(Decide Direction)



Apply Pattern
(Search / Insert / Delete)



Inorder for Sorted
Range for Validation



Practice & Master
All BST Problems

7 ADVANCED TREE PROBLEMS

Important Problems beyond Basics

💡 AHA = Key Insight

⚙️ Pattern = How to think

Σ Formula / Idea

🔗 Complexity

1. LOWEST COMMON ANCESTOR IN BINARY TREE (LC 236)

Example

```
graph TD
  3((3)) --> 5((5))
  3 --> 1((1))
  5 --> 6((6))
  5 --> 2((2))
  2 --> 7((7))
  2 --> 4((4))
  1 --> 0((0))
  1 --> 8((8))
  style 3 fill:#bbf7d0,stroke:#22c55e,stroke-width:2px
  style 5 fill:#bbf7d0,stroke:#22c55e,stroke-width:2px
  style 1 fill:#bbf7d0,stroke:#22c55e,stroke-width:2px
  style 6 fill:#bbf7d0,stroke:#22c55e,stroke-width:2px
  style 2 fill:#bbf7d0,stroke:#22c55e,stroke-width:2px
  style 7 fill:#bbf7d0,stroke:#22c55e,stroke-width:2px
  style 4 fill:#bbf7d0,stroke:#22c55e,stroke-width:2px
  style 0 fill:#bbf7d0,stroke:#22c55e,stroke-width:2px
  style 8 fill:#bbf7d0,stroke:#22c55e,stroke-width:2px
```

p = 5, q = 1 → LCA = 3

💡 AHA

Children report back.
If both left and right return non-null → current node is LCA.

⚙️ Pattern

Postorder (Left → Right → Root)

Σ Idea / Formula

```
If root = null or root = p or root = q → return root
left = recurse(left), right = recurse(right)
If left ≠ null and right ≠ null → return root
Else return left if not null else right
```

Complexity Time: $O(n)$ Space: $O(h)$

4. PATH SUM (LC 112 / 113)

Example (Sum = 22)

```
graph TD
  5((5)) --> 4a((4a))
  5 --> 8((8))
  4a --> 11((11))
  4a --> 7((7))
  11 --> 13((13))
  11 --> 8b((8b))
  8b --> 4b((4b))
  4b --> 1((1))
  style 5 fill:#fef3c7,stroke:#d97706,stroke-width:2px
  style 4a fill:#fef3c7,stroke:#d97706,stroke-width:2px
  style 11 fill:#fef3c7,stroke:#d97706,stroke-width:2px
  style 8b fill:#fef3c7,stroke:#d97706,stroke-width:2px
  style 4b fill:#fef3c7,stroke:#d97706,stroke-width:2px
  style 1 fill:#fef3c7,stroke:#d97706,stroke-width:2px
```

💡 AHA

Need root to leaf path.
Keep subtracting from target.
If leaf and target == 0 → valid path.

⚙️ Pattern

DFS (Preorder)

Σ Idea / Formula

2. CONSTRUCT BINARY TREE FROM TRAVERSALS (LC 105)

Example

```
Inorder : [9, 3, 15, 20, 7]
Preorder : [3, 9, 20, 15, 7]
```

```
graph TD
  3((3)) --> 9((9))
  3 --> 20((20))
  9 --> 15((15))
  20 --> 7((7))
```

💡 AHA

Preorder → Root comes first.
Inorder → Left and Right parts.
Build tree using indices.

⚙️ Pattern

Recursive (Divide & Conquer)

Σ Idea / Formula

1. First element in preorder = Root
2. Find root in inorder → split left & right
3. Recur for left part and right part

Complexity Time: $O(n)$ Space: $O(n)$

5. BUILD TREE FROM INORDER AND POSTORDER (LC 106)

Example

```
Inorder : [9, 3, 15, 20, 7]
Postorder : [9, 15, 7, 20, 3]
```

```
graph TD
  3((3)) --> 9((9))
  3 --> 20((20))
  9 --> 15((15))
  20 --> 7((7))
```

💡 AHA

Postorder → Root comes last.
Inorder → Left and Right parts.

⚙️ Pattern

Recursive (Divide & Conquer)

Σ Idea / Formula

1. Last element in postorder = Root
2. Find root in inorder → split left & right
3. Recur for left part and right part

Complexity Time: $O(n)$ Space: $O(n)$

3. SERIALIZE AND DESERIALIZE BINARY TREE (LC 297)

Example (Serialize)

```
graph TD
  1((1)) --> 2((2))
  1 --> 3((3))
  2 --> 4((4))
  3 --> 5((5))
```

Serialized:

```
[1,2,4,null,null,5,3,null,6]
(Level Order with nulls)
```

💡 AHA

Store structure + values.
Use null marker for missing nodes.

⚙️ Pattern

BFS (Level Order) or Preorder

Σ Idea / Formula

Serialize → Traverse and add values (add nulls)

Deserialize → Rebuild tree from values

Complexity Time: $O(n)$ Space: $O(n)$

6. BURNING BINARY TREE (LC 2385)

Example

```
graph TD
  1((1)) --> 2((2))
  1 --> 3((3))
  2 --> 4((4))
  2 --> 5((5))
  3 --> 6((6))
  5 --> 7((7))
  style 1 fill:#fee2e2,stroke:#dc2626,stroke-width:2px,color:#dc2626
  style 2 fill:#fee2e2,stroke:#dc2626,stroke-width:2px,color:#dc2626
  style 3 fill:#fee2e2,stroke:#dc2626,stroke-width:2px,color:#dc2626
  style 4 fill:#fee2e2,stroke:#dc2626,stroke-width:2px,color:#dc2626
  style 5 fill:#fee2e2,stroke:#dc2626,stroke-width:2px,color:#dc2626
  style 6 fill:#fee2e2,stroke:#dc2626,stroke-width:2px,color:#dc2626
  style 7 fill:#fee2e2,stroke:#dc2626,stroke-width:2px,color:#dc2626
```

Start Node = 5
Time to burn all nodes = 4

💡 AHA

Fire spreads in all 4 directions.
Convert tree to graph (undirected).
Do BFS from start node.

⚙️ Pattern

Graph + BFS

Σ Idea / Formula

1. Convert tree to undirected graph
2. BFS from start node
3. Count levels (time)

Complexity Time: $O(n)$ Space: $O(n)$


```
target == node.val
If leaf and target == 0 → true
Else check left and right
```

COMMON MENTAL MODELS

	Need parent first	Preorder
	Need sorted order	Inorder
	Need children first	Postorder
	Need level by level	BFS
	Path / Root to Leaf	DFS (Backtrack)
	Multiple directions	Graph + BFS

TREE BUILDING INDEX RULES

Preorder + Inorder

Preorder **Root** | Left | Right
Inorder Left | **Root** | Right

→ Use root from preorder

Inorder + Postorder

Inorder Left | **Root** | Right
Postorder Left | Right | **Root**

→ Use root from postorder

QUICK RECAP

- ✓ Traversal decides the way you think.
- ✓ Return type decides the recursion.
- ✓ Break into subproblems.
- ✓ Combine results.
- ✓ Handle base cases carefully.
- ✓ Draw the tree. Dry run.
- ✓ **That's 80% of the battle won!**



 **FINAL MANTRA** Understand the Problem → Choose Pattern → Draw Tree → Apply Template → Code → Dry Run → Optimize

8 TREES - FAANG ONE PAGE REVISION

Aha Moments • Patterns • When to use what

1 WHEN TO USE WHICH TRAVERSAL?

Goal / Need	Use This	Why?
Need parent first (decision at node)	Preorder	Process node before children
Need sorted order (BST problems)	Inorder	Left → Node → Right gives sorted order
Need children's information first	Postorder	Process children before node
Need level by level (each level together)	BFS (Level Order)	Visit nodes level by level
Need all paths (root to leaf)	DFS (Backtracking)	Go deep, explore all paths

2 RECURSION - GOLDEN TEMPLATE (DFS)

- 1 BASE CASE**
Handle null / edge case
- 2 FAITH**
Assume left & right give correct answer
- 3 RECUR LEFT**
Recur for left subtree
- 4 RECUR RIGHT**
Recur for right subtree
- 5 COMBINE**
Use left & right results, do required work
- 6 RETURN**
Return the final answer

```
Type solve(Node node) {
    if (node == null)
        return ...; // 1. Base Case

    Type left = solve(node.left); // 3
    Type right = solve(node.right); // 4

    // 5. Combine
    Type ans = ...;

    return ans; // 6
}
```

REMEMBER
Return type decides what you can compute.
Break every problem into left & right subproblems.

3 DFS PATTERN RECAP

Problem	Key Idea / Pattern
Max Depth	1 + max(left, right)
Same Tree	left && right
Invert Tree	Swap left & right
Diameter	Through node = left + right
Balanced Tree	left - right <= 1

4 BFS PATTERN RECAP

Problem	Key Idea / Pattern
Level Order	Queue + level size
Right Side View	Last node of each level
Zigzag Order	Left to Right / Right to Left toggle
Average of Levels	Sum / size of level
Level Order (N-ary)	Same BFS with N children

5 BST QUICK HITS

- ✓ Left Subtree < Node < Right Subtree
- ✓ Inorder of BST = Sorted Array
- ✓ Search / Insert = Compare & Go
- ✓ Delete = 3 cases
(0 child, 1 child, 2 children)
- ✓ Validate BST = Check range (min, max)
- ✓ Kth Smallest = Inorder (kth element)
- ✓ LCA in BST = Use BST property

6 COMMON TREE PROBLEM PATTERNS

Pattern	Think
Height / Depth	Add 1 + max(left, right)
Exists / Search	Return true if found in left or right
Count / Sum	Count/Sum from left + right + current
Path Problems	Carry path / sum while going down
Construct Tree	Use Inorder + (Pre/Post)order
Validate / Check	Use constraints / ranges while traversing

7 MENTAL MODELS (AHA MOMENTS)

- ★ **Preorder** → I do (parent) first
- ★ **Inorder** → I am in the middle (sorted for BST)
- ★ **Postorder** → I check results of my children first
- ★ **BFS** → I explore level by level
- ★ **DFS** → I go deep and backtrack
- ★ Every node can be a center of diameter
- ★ Tree is a connected acyclic graph
- ★ Remove null checks with base case
- ★ For BST → think in terms of range
- ★ Break problem → Solve small → Combine → Return

8 COMPLEXITY CHEAT SHEET

Operation / Problem	Time	Space
Traversal (DFS)	O(n)	O(h)
Traversal (BFS)	O(n)	O(w)
Search in BST	O(h)	O(1)
Insert in BST	O(h)	O(1)
Delete in BST	O(h)	O(h)
Height of Tree	O(n)	O(h)
Diameter	O(n)	O(h)

n = number of nodes, h = height, w = max width

9 TRAVERSAL VISUAL SUMMARY

Preorder (Root → Left → Right)	Inorder (Left → Root → Right)	Postorder (Left → Right → Root)	Level Order (BFS) (Level by Level)
graph TD A((1)):::root → B((2)) A → C((3)) B → D((4)) B → E((5)) C → F((6)) C → G((7)) classDef root fill:#fef08a,stroke:#ca8a04	graph TD A((1)):::root → B((2)) A → C((3)) B → D((4)) B → E((5)) C → F((6)) C → G((7)) classDef root fill:#bbf7d0,stroke:#16a34a	graph TD A((1)):::root → B((2)) A → C((3)) B → D((4)) B → E((5)) C → F((6)) C → G((7)) classDef root fill:#fde68a,stroke:#d97706	graph TD A((1)):::root → B((2)) A → C((3)) B → D((4)) B → E((5)) C → F((6)) C → G((7)) classDef root fill:#bfbdbf,stroke:#2563eb
Order: 1, 2, 4, 5, 3, 6, 7	Order: 4, 2, 5, 1, 6, 3, 7	Order: 4, 5, 2, 6, 7, 3, 1	Order: 1, 2, 3, 4, 5, 6, 7

10 FINAL MANTRA

- ✓ Understand the Problem
- ✓ Choose the Right Pattern
- ✓ Draw / Dry Run
- ✓ Write Clean Code
- ✓ Test All Edge Cases
- ✓ Optimize if needed

★ PRACTICE → ANALYZE →
REFLECT → IMPROVE → REPEAT

IMPORTANT REMINDERS

- Draw the tree
Always helps!
- Dry run
before coding
- Handle null
carefully
- Think recursively,
not iteratively
- Keep solving,
keep improving!